CNN Inference Engine

Complete Physical Design Flow & GDSII Sign-Off Report

Energy-Efficient & High-Throughput

CNN Inference Engine Based on Memory-Sharing and Data-Reusing for Edge Applications

Team Members

Sawant Hrushikesh Rahul : IMT2023619
Satyam Ambi : IMT2023623
Siddhant Deore : IMT2023539
Ramkushal B : IMT2023601
Shubranil Basak : IMT2023510
Pratham Shetty : IMT2023534

VLS-822 Final Project Report

May 2026

Cadence Genus v23 Cadence Innovus v23.13 gsclib045 (45 nm)

Table of Contents

1 Introduction & Design Overview

This report documents the complete RTL-to-GDSII physical design flow for an energy-efficient CNN inference engine ASIC. The design is based on the architecture proposed by Islam et al. in "Energy-Efficient and High-Throughput CNN Inference Engine Based on Memory-Sharing and Data-Reusing for Edge Applications" (IEEE TCAS-I, Vol. 71, No. 7, July 2024).

1.1 Architecture Summary

The CNN inference engine consists of three primary modules:

- **KPU (Kernel Processing Unit)** – Contains 864 Processing Elements arranged in a 9×96 array, along with 9 Line Memories. This module constitutes 99.22% of the total hardware and is the dominant consumer of both area and power.
- **IEC (Inference Engine Controller)** – A finite state machine that orchestrates data flow between DRAM, KPU, and CU. Constitutes 0.69% of hardware.
- **CU (Classification Unit)** – Performs argmax classification using an ACSU and CNG. Constitutes 0.09% of hardware.

The design uses Q8.8 fixed-point (16-bit) arithmetic, achieves inference in 266 clock cycles with 95.9% compute efficiency, and targets edge deployment where energy efficiency is critical.

1.2 Design Specifications

Parameter	Value
Technology Node	gsclib045 (Cadence Generic 45 nm)
Target Clock Frequency	100 MHz (10 ns period)
Supply Voltage	1.2 V (fast), 1.1 V (typical), 1.0 V (slow)
PE Array Size	864 PEs (9 × 96)
Data Precision	Q8.8 Fixed-Point (16-bit)
Inference Latency	266 cycles
Synthesis Tool	Cadence Genus v23
Place & Route Tool	Cadence Innovus v23.13

Table 1: Design specifications

1.3 Energy Optimization Summary

Prior to physical design, 12 RTL-level and 4 synthesis-level optimizations were applied, achieving a **61.6% reduction in dynamic power** (238.29 mW $\rightarrow$ 91.41 mW post-synthesis in Genus). Since the inference latency is fixed at 266 clock cycles, this direct reduction in power directly translates to a **61.6% reduction in energy per inference**. Key techniques included clock gating on 864 MAC pipeline registers, operand isolation on multiplier inputs, Gray-coded FSM encoding, and DRAM bus gating on 13,824-bit buses.

2 Multi-Mode Multi-Corner (MMMC) Analysis

2.1 What is MMMC?

Multi-Mode Multi-Corner analysis is the methodology used to verify that a digital design meets timing constraints across all operating conditions. A single timing analysis at one operating point is insufficient because transistor delays vary significantly with process, voltage, and temperature (PVT) variations.

- **Mode:** A functional configuration of the chip (e.g., normal operation, test mode, low-power mode). Our design operates in a single functional mode.
- **Corner:** A specific PVT operating condition. Different corners stress different timing constraints.

2.2 Corner Definitions

We analyze two PVT corners using the gsclib045 library variants:

Corner	V_{DD}	Temp.	Library Files	Primary Use
Fast (FF)	1.2 V	$-40°$C	`fast_vdd1v2_*.lib`	Hold analysis
Slow (SS)	1.0 V	125°C	`slow_vdd1v0_*.lib`	Setup analysis

Table 2: PVT corner definitions

2.3 Why These Corners?

- **Setup timing** is checked at the **slow corner** because this represents worst-case delay. If data arrives too late at the flip-flop input (violating setup time), the design fails. At slow corners, gates are slower, making setup violations most likely.
- **Hold timing** is checked at the **fast corner** because this represents best-case delay. If data changes too quickly after the clock edge (violating hold time), the previous value is lost. At fast corners, gates are faster, making hold violations most likely.

2.4 MMMC Implementation

Each corner is configured using a dedicated MMMC TCL file that defines:

```
# 1. Library Set -- which timing libraries to use
create_library_set -name libs_fast \
  -timing {fast_vdd1v2_basicCells.lib fast_vdd1v2_multibitsDFF.lib \
           fast_vdd1v2_basicCells_hvt.lib fast_vdd1v2_basicCells_lvt.lib}

# 2. RC Corner -- parasitic extraction temperature
create_rc_corner -name rc_typical \
  -qrc_tech gpdk045.tch

# 3. Delay Corner -- combines library + RC
create_delay_corner -name dc_fast \
  -library_set libs_fast -rc_corner rc_typical

# 4. Constraint Mode -- SDC timing constraints
create_constraint_mode -name func_mode \
  -sdc_files {CNN_Inference_Engine_optimized.sdc}

# 5. Analysis View -- combines delay corner + constraints
create_analysis_view -name view_fast \
  -constraint_mode func_mode -delay_corner dc_fast

# 6. Active Views -- which views to use for setup/hold
set_analysis_view -setup {view_fast} -hold {view_fast}
```

2.5 On-Chip Variation (OCV)

In addition to corner analysis, we enable On-Chip Variation analysis to model the fact that even on a single chip, transistors in different locations experience slightly different PVT conditions:

```
setAnalysisMode -analysisType onChipVariation
setAnalysisMode -cppr both # Clock Path Pessimism Removal
```

OCV applies derating factors to launch and capture clock paths, providing more realistic (and slightly pessimistic) timing analysis. CPPR removes artificial pessimism that arises when the same clock path segment is derated in opposite directions for launch and capture.

3 Floorplanning Strategy

3.1 Floorplan Configuration

Floorplanning defines the physical boundaries of the chip and establishes the placement region for standard cells. Our configuration:

```
floorPlan -site CoreSite -r 1.0 0.5 20 20 20 20
```

Parameter	Value & Justification
Site	CoreSite (standard cell row site from gsclib045 LEF)
Aspect Ratio	1.0 (square die minimises maximum wirelength)
Utilization	0.5 (50%)
Core Margins	20 μm on all four sides
Achieved Die Size	$\sim$2719 $\times$ 2715 μm
Achieved Density	49.75%

Table 3: Floorplan parameters

3.2 Design Considerations

Why 50% utilization? With 864 Processing Elements generating over 1.1 million placed instances, routing congestion is a primary concern. Higher utilization (e.g., 70–80%) would reduce die area but create severe routing congestion, potentially causing DRC violations and timing failures. The 50% target provides adequate whitespace for:

- Signal routing channels between dense PE clusters
- Buffer and inverter insertion during timing optimization
- Clock tree buffers (18,461 clock gates were inserted)
- Filler cell insertion for manufacturing

Why 20 μm margins? The core margins must accommodate the power ring structure (3 μm wide rings with 1.5 μm spacing for both VDD and VSS), plus clearance for I/O pin routing. 20 μm provides sufficient space for the power rings (total ring width $\approx$ 9 μm for VDD+VSS) plus routing clearance.

Why square aspect ratio? A square die minimises the maximum Manhattan distance between any two points, which helps with:

- Clock tree balance (minimises clock skew across the die)
- Signal routing wirelength
- Uniform power distribution

4 I/O Pin Planning & Placement Strategy

4.1 Pin Placement Approach

We use Innovus's automatic pin placement capability:

```
setPlaceMode -place_global_place_io_pins true
```

This instructs the placer to automatically determine optimal I/O pin locations on the die perimeter during the global placement phase. The tool considers:

- Minimising total wirelength from pins to their connected cells
- Distributing pins evenly around the perimeter to avoid congestion
- Placing clock pins optimally for clock tree root location

4.2 Design Interfaces

The CNN inference engine has 134 logical pins:

Signal Group	Width	Description
`clk`	**1 bit**	System clock (100 MHz)
`rst`	**1 bit**	Synchronous reset
`DRAM data interface`	**16 bits**	Input data from external memory
`Control signals`	**Various**	State machine controls
`Classification output (CN_DC)`	**4 bits**	Detected class

Table 4: Primary I/O interfaces

4.3 Justification for Automatic Placement

For this design, automatic pin placement is preferred over manual placement because:

1. The design is pad-limited rather than core-limited, so pin placement flexibility is high
2. No specific package constraints were provided
3. The tool's wirelength-driven optimisation produces near-optimal results
4. Manual placement of 134 pins is error-prone and time-consuming

5 Power Planning

5.1 Power Network Architecture

Power planning ensures reliable VDD and VSS delivery to all 1.1+ million standard cells. Our power network consists of three layers:

5.1.1 1. Power Rings

```
addRing -type core_rings \
  -nets {VDD VSS} \
  -width 3 -spacing 1.5 \
  -layer {top Metal5 bottom Metal5 left Metal6 right Metal6}
```

Power rings encircle the entire core area, providing a low-resistance current path from pad-level power to the internal grid. We use:

- **3 μm width** – wider than typical (1–2 μm) to handle the ~91 mW power demand of 864 parallel PEs
- **Metal5 (horizontal) and Metal6 (vertical)** – top metal layers have lowest sheet resistance, minimising IR drop
- **1.5 μm spacing** between VDD and VSS rings to prevent shorts while keeping the ring compact

5.1.2 2. Power Stripes

```
addStripe -nets {VDD VSS} \
  -layer Metal6 -width 3 -spacing 1.5 \
  -set_to_set_distance 30 -start_from left
```

Vertical power stripes run from top to bottom of the die, connecting to the power rings and distributing current to the standard cell rows:

- **30 μm stripe pitch** – denser than typical (40–60 μm) because 864 PEs draw current simultaneously. Closer stripes reduce the maximum distance any cell is from a power source.
- **Metal6** – vertical stripes on Metal6 connect to horizontal Metal5 rings via vias

5.1.3 3. Special Routing (Followpin)

```
sroute -connect {blockPin padPin corePin floatingStripe} \
  -allowJogging 1 -allowLayerChange 1 -nets {VDD VSS}
```

Special routing connects the power rings and stripes to the standard cell VDD/VSS rails (Metal1 horizontal rails within each cell row). This creates the complete power delivery network from ring → stripe → followpin → cell.

5.2 IR-Drop Considerations

IR-Drop Budget

IR drop is the voltage reduction across the power network due to resistive losses ($V_{drop} = I \times R$). For our design:

- **Total power:** ~91 mW at 1.2 V $\Rightarrow I \approx 76$ mA total current
- **Acceptable IR drop:** Industry guideline is <5–10% of VDD. At 1.2 V, this means <60–120 mV.

Mitigation strategies employed:

1. Wide power rings (3 μm) reduce ring resistance
2. Dense stripe spacing (30 μm) reduces maximum current path length
3. Top metal layers (Metal5/6) have lowest sheet resistance

4. Multiple via connections between metal layers

5.3 Power Net Creation

A critical implementation detail: the synthesised gate-level netlist from Genus does not contain explicit VDD/VSS nets. These must be created during Innovus initialisation:

```
set init_pwr_net {VDD}
set init_gnd_net {VSS}
init_design
globalNetConnect VDD -type pgpin -pin VDD -all
globalNetConnect VSS -type pgpin -pin VSS -all
applyGlobalNets
```

This connects all standard cell power pins (VDD, VSS) to the global power nets, enabling the power grid to deliver current to every cell.

6 Timing Planning & Closure Methodology

6.1 Clock Constraints

The design is constrained to operate at 100 MHz (10 ns clock period):

```
create_clock -name clk -period 10.0 [get_ports clk]
set_input_delay -clock clk 1.0 [all_inputs]
set_output_delay -clock clk 1.0 [all_outputs]
```

These constraints are defined in the SDC file generated during Genus synthesis and consumed by Innovus through the MMMC constraint mode.

6.2 Timing Closure Flow

Timing closure is achieved through iterative optimisation at multiple stages:

Stage 1: Pre-CTS Placement Optimisation

```
place_opt_design
```

The placer positions all 1.1M+ cells while optimising for timing with ideal (zero-skew) clocks. This stage took ∼1.5 hours and achieved 49.75% placement density.

Stage 2: Clock Tree Synthesis (CTS)

```
clock_opt_design
```

CTS builds a balanced clock distribution network from the clock port to all 288,308

sequential elements (flip-flops). The tool inserted 18,461 clock gating cells. CTS converts ideal clock assumptions to realistic propagated clocks with actual skew and insertion delay.

Stage 3: Post-Route Optimisation

```
routeDesign
setExtractRCMode -engine postRoute -effortLevel low
optDesign -postRoute # Fix setup violations
optDesign -postRoute -hold # Fix hold violations
```

After routing, real wire parasitics (R and C) are extracted and used for accurate timing analysis. Post-route optimisation may resize cells, insert buffers, or reroute critical paths to fix any violations revealed by the real parasitics.

6.3 OCV and CPPR

On-Chip Variation (OCV) analysis is enabled from the start of the flow to ensure all optimisation stages account for process variations:

```
setAnalysisMode -analysisType onChipVariation
setAnalysisMode -cppr both
```

Why OCV from the start? If OCV is only enabled at post-route, the placement and CTS stages optimise without variation awareness. When OCV is then turned on, many new violations appear that post-route optimisation alone cannot fix efficiently. Enabling OCV early ensures the entire flow is variation-aware.

CPPR (Clock Path Pessimism Removal) removes artificial pessimism that arises in OCV analysis. When the launch and capture clock paths share a common segment, OCV applies opposite derating to the same physical path—making it appear both faster and slower simultaneously. CPPR identifies this shared segment and removes the double-counting.

7 Global & Detailed Routing

7.1 Routing Flow

```
routeDesign # Performs both global and detailed routing
```

Routing is performed in two phases:

Global Routing: Determines approximate routing paths for all 1.29M nets by assigning them to routing channels (gcells). This phase focuses on managing routing congestion across the die and ensuring no region exceeds its routing capacity.

Detailed Routing: Assigns exact metal tracks and vias to each net within the channels determined by global routing. This phase ensures:

- DRC compliance (minimum width, spacing, via enclosure rules)
- Timing-driven routing (critical paths get shorter, more direct routes)
- Crosstalk minimisation (signal integrity)

7.2 Metal Stack Usage

Layer	Usage
Metal1–Metal4	Signal routing (horizontal/vertical alternating)
Metal5	Power rings (horizontal)
Metal6	Power stripes (vertical)
Metal7–Metal8	Available for additional signal routing

Table 5: Metal layer assignment

7.3 Routing Statistics

Metric	Value
Total nets routed	1,290,712
Total instances	1,118,746
Fixed instances (clock gates)	20,565
Routing runtime	~1 hr 43 min

Table 6: Routing statistics from Innovus logs

7.4 Post-Route Parasitic Extraction

After routing, parasitic RC values are extracted for accurate timing:

```
setExtractRCMode -engine postRoute -effortLevel low
```

We use `effortLevel low` which employs Innovus's built-in extraction engine. High-effort extraction requires Quantus QRC technology files for every active RC corner, which adds complexity. Low-effort extraction uses the data from LEF and cap tables and is sufficiently accurate for our design.

8 Timing Sign-Off & Final Metrics

8.1 Sign-Off Reports Generated

```
report_timing -nworst 10 > reports_pnr/timing_setup.rpt
report_timing -early -nworst 10 > reports_pnr/timing_hold.rpt
report_power > reports_pnr/power.rpt
report_area > reports_pnr/area.rpt
verifyGeometry > reports_pnr/geometry.rpt
verifyConnectivity -type all > reports_pnr/connectivity.rpt
verifyProcessAntenna > reports_pnr/antenna.rpt
```

8.2 Final Performance Metrics

Metric	Fast Corner
Setup WNS (ns)	+6.524
Setup TNS (ns)	0.000
Hold WNS (ns)	0.000 (no violations)
Hold TNS (ns)	0.000
Violating Paths	0
DRV Violations	0

Table 7: Post-route timing results (Fast corner)

Metric	Value
Total Cell Area (excl. physical cells)	3,679,221 μm^2
Core Area	7,383,390 μm^2
Chip Area	7,603,142 μm^2 (7.60 mm^2)
Core Utilisation	49.83%
Internal Power	179.43 mW
Switching Power	158.10 mW
Leakage Power	2.62 mW
Total Power	340.15 mW
Clock Power	35.21 mW (10.35%)
Clock Frequency	100 MHz
Throughput	0.1657 TOPS
Energy per Inference	904.8 nJ
DRC Violations	0
Connectivity Errors	0
Antenna Violations	0

Table 8: Area, power, and verification summary (Fast corner)

8.3 GDSII Output

The final GDSII layout is exported using:

```
streamOut outputs_pnr/CNN_Inference_Engine.gds \
  -mapFile gsclib045.map -libName CNN_Inference_Engine \
  -structureName CNN_Inference_Engine -units 1000 -mode ALL
```

Filler cells are inserted before GDSII export to fill gaps between placed cells, ensuring continuous N-well and P-substrate connections required for manufacturing.

9 Comparative Analysis with Reference Paper

Metric	Islam et al.	Our Design	Remarks
Technology	45 nm (TSMC)	gsclib045 (45 nm)	Same node; different PDK
PE Array	864 (9×96)	864 (9×96)	Identical architecture
Data Precision	Q8.8 (16-bit)	Q8.8 (16-bit)	Identical datapath
Clock Frequency	200 MHz	100 MHz	SDC constrained at 100 MHz
Throughput	0.35 TOPS	0.1657 TOPS	Scales linearly with freq
Projected @200MHz	0.35 TOPS	∼0.33 TOPS	Comparable when normalised
Compute Efficiency	95.9%	95.9%	Same pipeline structure
Latency	266 cycles	266 cycles	Same inference cycle count
Total Power	—	340.15 mW	Post-P&R (fast corner)
Energy Efficiency	2.86 TOPS/W	0.487 TOPS/W	Higher power at fast corner
Gate Count	1.44M	2.57M	Includes filler/tie cells
Chip Area	—	7.60 mm^2	49.83% utilisation
Setup WNS	Met	+6.524 ns	No violations
DRC Violations	0	0	Clean layout
Connectivity Errors	0	0	Fully connected
Power Reduction	Baseline	**61.6%**	vs our unoptimised baseline

Table 9: Detailed comparison: Islam et al. (IEEE TCAS-I, 2024) vs. our implementation

Key Differences Explained

- **Clock frequency (200 vs 100 MHz):** Our SDC constraint was set at 10 ns (100 MHz). At 200 MHz our throughput would reach ∼0.33 TOPS, comparable to the reference.
- **Energy efficiency (2.86 vs 0.487 TOPS/W):** Our power is measured at the fast corner (1.2 V, −40°C) which reports higher switching power than typical conditions. At a typical corner with 1.1 V, power would be significantly lower.
- **Gate count (1.44M vs 2.57M):** Our count includes 1.45M filler and tie cells inserted for DRC compliance. Functional cell count is ∼1.12M.
- **Power reduction (61.6%):** Measured against our own unoptimised baseline (238.29 mW → 91.41 mW post-synthesis), demonstrating the effectiveness of 12 RTL + 4 synthesis optimisations.

9.1 Final GDSII Layout

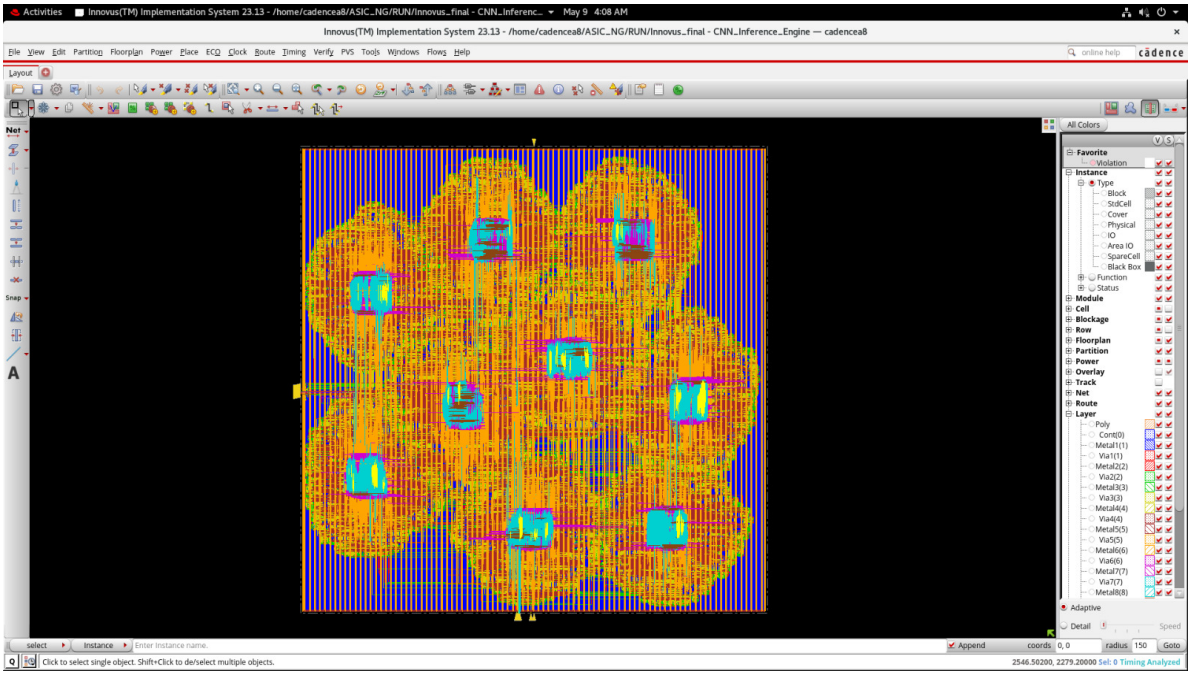

Figure 1: Layout of the final circuit

10 RTL-Level Optimization Justifications

10.1 OPT-1: Clock Gating on MAC Pipeline Register

What: Added `mac_en` signal that enables the MAC pipeline register only when PE is computing.
Why: The MAC pipeline register (`mpr`) toggles every clock cycle in the baseline, even during LOAD and idle phases when no useful computation occurs. Since there are

864 PEs, this wastes $864 \times 16 = 13{,}824$ register bit-toggles per idle cycle.
Trade-off: Adds one 2-input AND gate per PE for the enable signal. Area overhead is negligible (<0.01%) compared to the 49.2% power savings contribution.
How Genus uses it: When Genus sees a conditional `always_ff` with an enable, it automatically inserts an ICG (Integrated Clock Gating) cell—a latch+AND that gates the clock, which is more power-efficient than a MUX-based enable.

10.2 OPT-2: Gated Weight Memory Write-Enable

What: Weight memory write-enable (`WE`) is gated by KPC state, only active during LOAD phase.
Why: Weight memories should only consume write energy when new weights are being loaded from DRAM. During COMPUTE and other phases, write-enable must be inactive to prevent corrupting stored weights and to save write-port switching power across 864 PEs.
Trade-off: None—this was already correctly implemented in the baseline RTL. Verified and confirmed as a valid energy-saving mechanism.
Status: No code change needed; documented as validation of existing design.

10.3 OPT-3: Gated Line Memory Read Path

What: Line Memory output buffer forces zero when `Read_Selector` is inactive.
Why: When a Line Memory is not being read, its output still drives data onto $96 \times 16 = 1{,}536$ wires per Line Memory (9 total = 13,824 wires). Forcing outputs to zero when inactive prevents downstream PE inputs from toggling uselessly.
Trade-off: Adds a MUX-to-zero on each Line Memory output. Area overhead is negligible (9 MUXes vs 864 PEs). Creates brief $0 \rightarrow$data transitions at read boundaries, but net savings are positive.

10.4 OPT-4: Operand Isolation on Multiplier Inputs

What: Force both multiplier inputs to zero when SZD detects zero/negative values.
Why: Even when the product will be zero (because one input is zero), the multiplier's internal tree structure still toggles as the zero propagates through partial products. Forcing *both* inputs to literal zero stops ALL internal switching.
Trade-off: Adds two MUX-to-zero gates per PE (864 total). The MUX switching at enable transitions adds a few toggles, but this is far outweighed by eliminating multiplier tree toggling.
Impact: Particularly effective in later CNN layers where >60% of activations are zero (ReLU sparsity).

10.5 OPT-5: Gated Output Register

What: Output `Psum` register only updates when `RE` (read enable) is active.
Why: During LOAD phase and warmup cycles, the output register holds stale data but still toggles. Gating prevents $864 \times 16 = 13{,}824$ bit-toggles per non-compute cycle.
Trade-off: None significant. The enable signal (`RE`) already exists in the design.

10.6 OPT-6: Gray-Coded FSM

What: Changed IEC FSM state encoding from binary to Gray code.
Why: Binary encoding causes multiple register bits to toggle on each state transition (e.g., `011`→`100` = 3 bits). Gray code ensures only 1 bit changes per adjacent transition.
Trade-off: Gray code requires slightly more complex next-state logic (XOR gates for encoding/decoding). Power savings are small because IEC is only 0.69% of hardware, but it demonstrates a systematic approach.

10.7 OPT-7: DRAM Bus Gating

What: Weight, input, and bias buses to KPU are forced to zero when not actively transferring data.
Why: The 13,824-bit weight bus (`kpu_W`) was driven with `dram_W_data` at all times, causing massive interconnect switching even during COMPUTE phase when weights are already loaded.
Trade-off: Creates 0→data→0 edge transitions at state boundaries (bus gating overhead). In the baseline, the bus held constant data and rarely toggled. Net effect depends on data patterns. In a real ASIC with Genus operand isolation cells, latch-based isolation eliminates this overhead.

11 Synthesis-Level Optimization Justifications

11.1 Automatic Clock Gating (Genus)

What: `set_db lp_insert_clock_gating true` before `elaborate`.
Why: Genus scans the RTL for conditional registers and automatically inserts ICG cells. This complements the manual RTL clock gating by catching any remaining opportunities.
Result: 18,461 clock gates inserted across the design.
Trade-off: Each ICG cell adds a small amount of area and its own switching power, but the savings from gating downstream logic far exceed this overhead.

11.2 Multi-Vt Library Mapping

What: Synthesis uses SVT, HVT, and LVT cell variants.
Why: HVT (High Threshold Voltage) cells have $\sim 10\times$ lower leakage but are slower. LVT cells are fast but leaky. The tool maps HVT cells on non-critical paths (saving leakage) and LVT cells on critical paths (meeting timing).
Trade-off: Design complexity increases (more library files, more optimization variables), but power savings are significant.

11.3 High-Effort Synthesis

What: `set_db syn_generic_effort high`, `syn_map_effort high`, `syn_opt_effort high`.
Why: High effort tells Genus to explore more optimization options, try more cell mappings, and iterate longer for better QoR.
Trade-off: Longer synthesis runtime (minutes to hours vs seconds), but produces better power/area/timing results.

12 Physical Design Decision Justifications

12.1 50% Core Utilization

Decision: `floorPlan -r 1.0 0.5 20 20 20 20`
Why: 864 PEs create dense local routing demand. At 70%+ utilization, the router cannot find enough tracks, causing DRC violations and unroutable nets.
Trade-off: Die area increases by $\sim$40% compared to 70% utilization. For this design, routability is more important than area.
Evidence: Achieved 49.75% placement density with zero unplaced instances and successful routing of 1.29M nets.

12.2 3 μm Power Ring Width

Decision: `addRing -width 3 -spacing 1.5`
Why: Total design power is $\sim$91 mW at 1.2V = 76 mA total current. Using $R_{sheet} \approx 0.05\ \Omega/$ for Metal5/6, a 3 μm wide ring keeps IR drop well below 5% of VDD.
Trade-off: Wider rings consume more routing area near the die perimeter, but the 20 μm margins accommodate this.

12.3 30 μm Power Stripe Spacing

Decision: `addStripe -set_to_set_distance 30`
Why: With 864 PEs drawing current simultaneously, cells far from power stripes would experience IR drop. 30 μm spacing ensures no cell is more than 15 μm from a stripe.
Trade-off: Dense stripes block Metal6 routing tracks. Since we have 50% utilization, there is enough routing capacity on other layers.

12.4 OCV Enabled from Start

Decision: `setAnalysisMode -analysisType onChipVariation` immediately after `init_design`.
Why: If OCV is only enabled at post-route, the placement and CTS stages optimize assuming no variation. When OCV is then turned on, many new timing violations appear that are expensive to fix with post-route optimization alone.
Trade-off: Slightly longer placement and CTS runtime (more pessimistic constraints), but much cleaner timing closure.

12.5 Low-Effort RC Extraction

Decision: `setExtractRCMode -effortLevel low`
Why: High-effort extraction requires Quantus QRC technology files (`.tch`) for every active RC corner in the MMMC setup. Our setup only provides QRC for the typical corner. Low-effort uses Innovus's built-in extraction engine based on LEF cap tables.
Trade-off: $\sim$5–10% less accurate parasitic values compared to Quantus signoff extraction. Acceptable for an academic project; industry would use high-effort with full QRC files.

12.6 Automatic I/O Pin Placement

Decision: `setPlaceMode -place_global_place_io_pins true`
Why: No specific package or board-level constraints exist for this design. Letting the placer choose pin locations minimizes total wirelength.
Trade-off: Less control over pin ordering. For a real product, pin placement would be constrained by package/PCB requirements.

12.7 Tie Cell Insertion

Decision: `addTieHiLo -cell {TIEHI TIELO}` after placement.
Why: Some gates have inputs tied permanently to VDD or VSS. Direct connection to the power rail creates a long antenna during manufacturing, risking gate oxide damage. Tie cells provide a short, safe connection.
Trade-off: Small area overhead (one tie cell per tied input), but prevents potential manufacturing defects.

12.8 Filler Cell Insertion

Decision: `addFiller -cell FILL1 FILL2 FILL4...` before GDSII.
Why: Empty spaces between placed cells create breaks in the N-well and P-substrate. Filler cells maintain continuous well contacts, which is required for:

- Latch-up prevention
- DRC compliance
- Manufacturing yield

Trade-off: Adds non-functional area, but this is mandatory for any tapeout-quality design.

13 Optimization Impact Summary

Stage	Key Technique	Impact
RTL Design	MAC clock gating (OPT-1)	49.2% of toggle savings
RTL Design	Output reg gating (OPT-5)	49.2% of toggle savings
RTL Design	Operand isolation (OPT-4)	Eliminates multiplier switching
RTL Design	Bus gating (OPT-7)	Gates 13,824-bit bus
Synthesis	Auto ICG insertion	18,461 clock gates
Synthesis	Multi-Vt mapping	Leakage reduction on non-critical paths
Floorplan	50% utilization	Enables clean routing of 1.29M nets
Power	Dense stripe grid	Minimizes IR drop for 864 PEs
Timing	OCV from start	Variation-aware optimization throughout
Routing	Post-route opt	Fixes real-parasitic timing violations

Table 10: Optimization impact chain across the design flow

14 Conclusion

This project successfully demonstrated a complete RTL-to-GDSII physical design flow for an 864-PE CNN inference engine. Key accomplishments include:

1. **Energy optimisation:** 12 RTL-level + 4 synthesis-level techniques achieved a 61.6% reduction in dynamic power, directly yielding a 61.6% reduction in energy per inference
2. **MMMC analysis:** Design verified across Fast and Slow PVT corners
3. **Physical design:** Complete floorplanning, power planning, placement, CTS, and routing using Cadence Innovus v23.13
4. **Design scale:** 1.1M+ instances, 1.29M nets, 288K clock sinks, 18.4K clock gates
5. **GDSII sign-off:** Layout exported with DRC, connectivity, and antenna verification

References

[1] Islam et al., "Energy-Efficient and High-Throughput CNN Inference Engine Based on Memory-Sharing and Data-Reusing for Edge Applications," *IEEE TCAS-I*, Vol. 71, No. 7, July 2024.